

CPG Analytics — Project Context (memory file)

Purpose: this file lets me (Claude) recover full context at the start of any session.

Read this first. It summarizes WHAT we're building, the GOVERNING principles, and WHERE things live.

Last updated: 2026-06-29 (consistency pass)

Current phase: SPEC PACKAGE (for handoff to coding team)

Producing `spec/` — versioned contracts (typed fields + JSON Schema + examples + CI validation). `ux.md` = thinking; `spec/` = what devs build against.

Locked schema decisions (v0.1): (1) one schema per concept, common to all levels — differences are field values not structure; (2) cards are semantic, layout is a separate section grammar; (3) cards are thin, reference central registries; (4) catalog slot defs are shared, cards carry only {slot, default, ask_priority}.

SPEC PACKAGE COMPLETE (DRAFT v0.1, 2026-06-22). All files written + cross-consistent:

`00-architecture.md` , `10-persona-card.schema.md` (+ `job_description`),
`11-catalog.schema.md` (v0.2 real CPG slots), `12-metric-registry.md` ,
`13-insight-rule.schema.md` , `14-section-grammar.schema.md` , `20-resolution.md` , `README`.

Examples: `asm.card`, `nsm.card`, catalog (9 slots), metric-registry (12), insight-rules (10), `nsm.home`. See [ux.md](#) Milestone 3.

CONSISTENCY PASS (2026-06-29): fixed cross-file drift. (1) insight-rules count corrected 11→10. (2) ASM `operating_grain` corrected `distributor` → `area` (matches role name + every rule's `applies_to`). (3) Action model clarified: the persona card owns the authoritative, role-appropriate insight→action mapping via `actions[].from_insight` ; `rule.actions` (13) are advisory defaults. Added card actions so every `insight_type` has a role action. (4) Added `chart.trend_grain` , priority buckets {critical|high|medium|low}, and two card↔rule CI checks. See [ux.md](#) Milestone 4.

DECIDED Q1 (assistant scope): `scope = persona job_description` . In the job → handle; outside → decline. Ignore emotional framing. Job = TOPIC only; RBAC = data access (separate).
 DECIDED Q2 (geo depth): cap by RESULT SIZE, not hierarchy level. Top-N allowed at any depth; big flat pulls require narrowing. "2-3 levels" = default home depth, not a wall. (spec/20 §5)

Phase 1 = defaults + confirm (no habit learning yet). Next: team review, then discussion on tenant overrides, scoring weights, question-bank location, LLM guardrails, first persona to build.

What we're building

A **governed CPG (consumer packaged goods) analytics experience** that walks a business user through four cognitive stages, each backed by a layer of the app:

Stage	Question	Layer
Descriptive	What happened?	Dashboard
Diagnostic	Why did it happen?	Insights + Drilldown
Prescriptive	What should I do next?	Recommended Actions
Conversational	Let me investigate further	Chat

All four share **one user context** and **one RBAC security scope**.

The one governing principle

Facts are deterministic. Language is intelligent.

- Cube.js + SQL + Python compute every number, ranking, threshold breach, confidence score, and priority score.
- The LLM ONLY narrates, explains, suggests wording, classifies intent, and converses.
- The LLM must NEVER invent metric values, bypass RBAC, or present a "possible reason" as a proven reason.

Current state of the real system

- User HAS: SQL fact tables + a working Cube.js semantic layer.
- User HAS DESIGNED: role-based YAML templates describing each persona's home experience.
- User WANTS: an engine/agent that reads the YAML, queries Cube, runs deterministic insight rules, scores/ranks, attaches actions + suggested questions, optionally LLM-enriches, and stores a prepared per-user "home payload" for fast login render.

Roles in scope

Sales Head, National Sales Manager, Regional Sales Manager, Area Sales Manager, Brand Manager, Marketing Manager, Business Analyst.

The 10-layer architecture (from design docs)

1. Data foundation (fact + dimension + user/security tables)
2. Semantic + RBAC (Cube.js; every query filtered by securityContext)
3. Personalization core (resolve user_context, map role -> template)
4. YAML template layer (business structure only, never UI/CSS)
5. Scheduler / experience generator (11-step per-user pipeline -> stored payload)
6. Insight engine (Risks / Opportunities / Anomalies / Drivers)
7. Confidence + Priority scoring (BOTH deterministic; never mixed)
8. LLM layer (narrative + intent only; strict guardrails)
9. Login experience (fetch stored payload, render)
10. Chat / investigation (suggested-question fast path vs free-text NLU) + feedback loop

Business model — THE SECRET SAUCE (established 2026-06-08)

B2B SaaS sold to CPG companies. Product ships with **vendor-curated Persona Cards** (SO, ASM, RSM, NSM, Brand Mgr, Marketing Mgr, Business Analyst). Each card pre-encodes CPG domain knowledge: sales hierarchy, KPIs, insight types, curated question bank, and a fully pre-loaded preference catalog with industry-correct defaults. Buyers just assign employees to personas → instant correct experience, cold start is never cold.

Three-tier resolution: Persona Card (vendor IP) → Tenant Binding (buyer data + RBAC) → Individual Model (learned per-person). Each tier overrides the one above, within RBAC.

User memory model = 5 layers: Identity (given/deterministic) · Preference (declared) · Behavioral (observed) · Episodic (history) · Working (session). Given identity is NEVER inferred; learned model is always reversible.

Curated Catalog principle: map every disambiguation slot to metrics/values; default most from role; ask a hard-capped 2-3 questions at cold start. Clarification lifecycle: ask → default → assume.

See `ux.md` for full detail + Milestone 1.

Open design questions (not yet resolved)

1. Stored-payload **freshness model** (when scheduler runs; trust cache vs revalidate data_as_of).
2. Where **thresholds & scoring weights** live (per-template? global? per-customer/tenant?).
3. **Two YAML schemas** in the docs need reconciling into one canonical grammar.
4. **Insight rules registry** (shared) vs per-template rules — recommend shared registry.
5. First deliverable = thin **vertical slice** (one role, real Cube query, 2-3 insights, scoring, 1 narrative, stored, rendered).

Source design docs (read-only reference, in user's Obsidian vault)

- `/Users/shyamsundarnarayanan/Documents/Obsidian/CPG Analytics/App Map.md`
- `/Users/shyamsundarnarayanan/Documents/Obsidian/CPG Analytics/CPG Analytics App – Core Goal and Experience Summary.md`

Working files (this project)

- `ux.md` — product & UX discussion + milestone snapshots
- `todo.md` — todo list; items added on "add to todo" command, stamped with date+time
- `open-questions.md` — running list of open PRODUCT questions to decide (e.g. Q1: Assistant scope)
- `spec/` — the contract package the coding team builds from (see its README)
- `PROJECT_CONTEXT.md` — this file

Conventions the user set

- We discuss UX/product FIRST, before any code.
- On reaching a milestone: snapshot it into `ux.md` with date + time.

- On the command "add to todo": append the item to `todo.md` with current date + time.